ΕΡΓΑΣΤΗΡΙΟ ΛΕΙΤΟΥΡΓΙΚΩΝ ΣΥΣΤΗΜΑΤΩΝ · 2η ΕΡΓΑΣΤΗΡΙΑΚΗ ΑΣΚΗΣΗ

Lunix:TNG

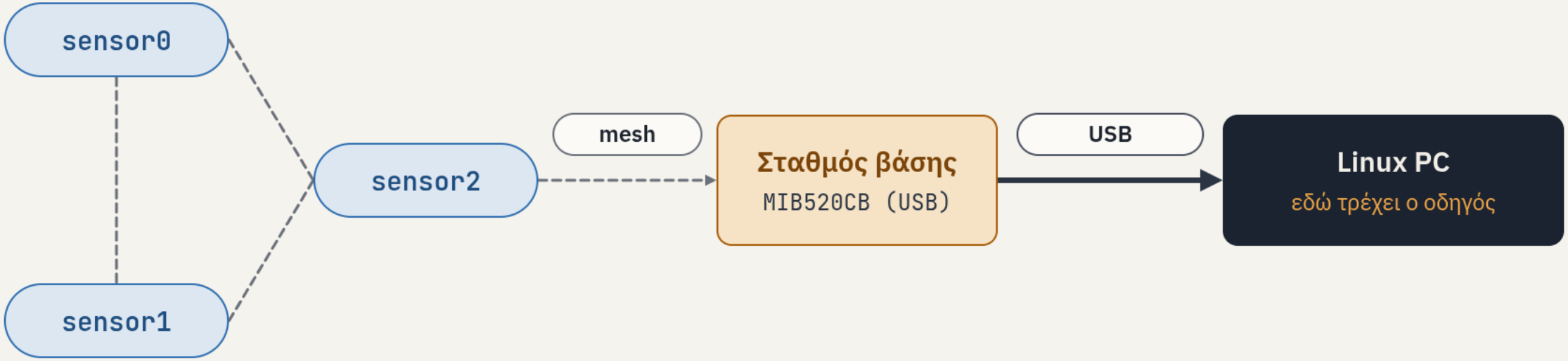
Ένας οδηγός συσκευής για ασύρματο δίκτυο αισθητήρων στο Linux

ΣΗΜΜΥ · ΕΜΠ

/dev/lunix0-temp

ΤΟ ΠΡΟΒΛΗΜΑ

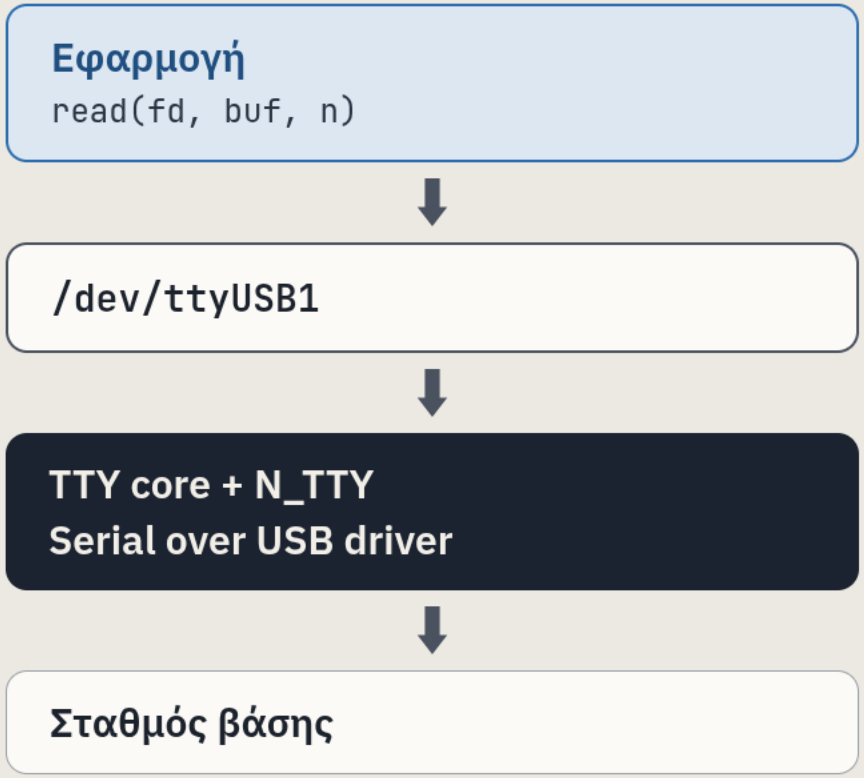
Ένα πραγματικό δίκτυο αισθητήρων



Τάση μπαταρίας
`batt`

Θερμοκρασία
`temp`

Φωτεινότητα
`light`



ΤΟ ΠΡΟΒΛΗΜΑ

Υπάρχει ήδη οδηγός. Γιατί δεν αρκεί;

01 **Όλα σε ένα αρχείο:** πακέτα από όλους τους αισθητήρες και όλες τις μετρήσεις, ανακατεμένα και σε δυαδική μορφή.

02 **Μία διεργασία:** ό,τι διαβάσει μία, δεν το βλέπει άλλη.

03 **Χωρίς λεπτομερή δικαιώματα:** ή βλέπεις τα πάντα ή τίποτα, όχι ανά αισθητήρα ή μέτρηση.

Η ΛΥΣΗ

Ένα αρχείο συσκευής ανά αισθητήρα και μέτρηση

```
$ ls /dev/lunix*
/dev/lunix0-batt  /dev/lunix0-temp  /dev/lunix0-light
/dev/lunix1-batt  /dev/lunix1-temp  /dev/lunix1-light
/dev/lunix2-batt  /dev/lunix2-temp  /dev/lunix2-light
... μέχρι τον αισθητήρα 15

$ cat /dev/lunix1-temp
26.734
26.832
^C
```

Αναγνώσιμες τιμές

Δεκαδικός αριθμός, όχι bytes πρωτοκόλλου.

Πολλοί αναγνώστες

Κάθε διεργασία βλέπει κάθε νέα μέτρηση.

Δικαιώματα ανά αρχείο

Τα γνωστά chmod / chown του συστήματος αρχείων.

Η ΛΥΣΗ

Πώς θα δουλεύει όταν τελειώσετε

```
user@vm:~/lunix-tng$ make
  CC [M]  lunix-chrdev.o  ...  LD [M]  lunix.ko
user@vm:~/lunix-tng$ su -
root@vm:/home/user/lunix-tng# insmod ./lunix.ko
root@vm:/home/user/lunix-tng# ./mk-lunix-devs.sh
root@vm:/home/user/lunix-tng# ./lunix-attach /dev/ttyS1
Line discipline set on /dev/ttyS1, press ^C ...

user@vm:~$ cat /dev/lunix2-temp
27.791
27.791
27.693
^C
```

make: χτίζει το lunix.ko, τους πίνακες αναζήτησης και το lunix-attach.

insmod: φορτώνει το module στον πυρήνα (ως root).

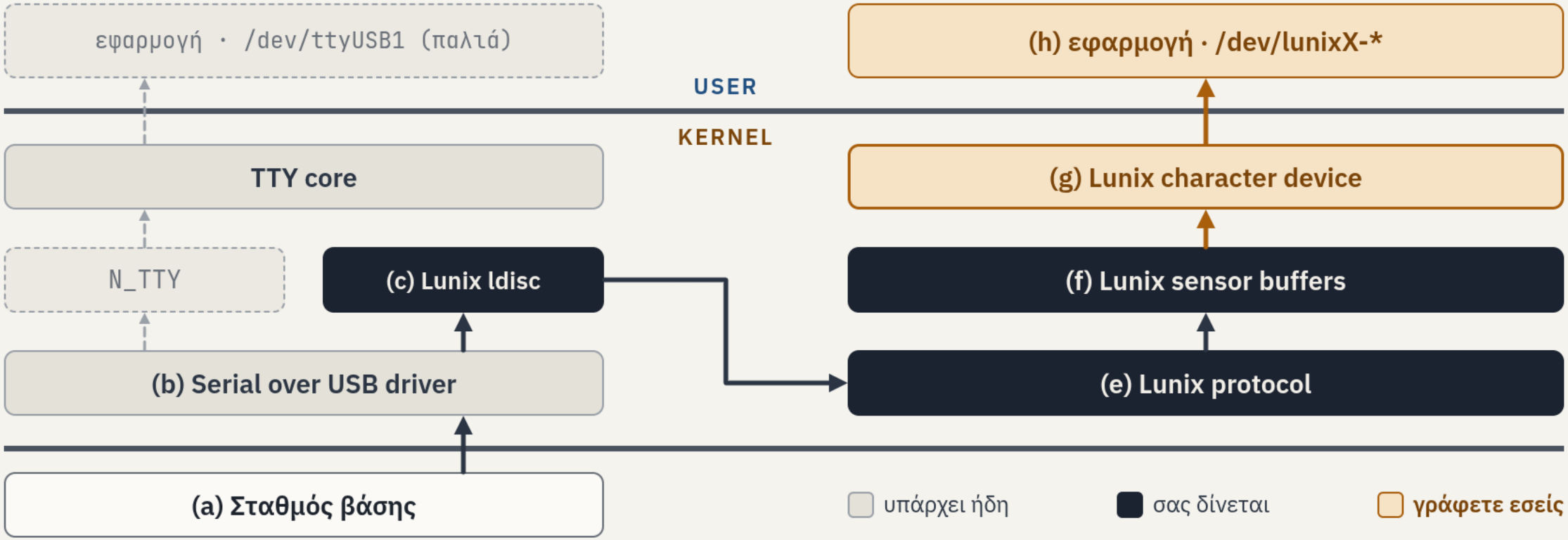
mk-lunix-devs.sh: φτιάχνει τους κόμβους στο /dev με mknod.

lunix-attach: «κουμπώνει» το Lunix στη σειριακή θύρα. Μένει ανοιχτό όσο θέλετε δεδομένα.

cat: μια τιμή ανά νέα μέτρηση· ανάμεσα, η cat κοιμάται.

ΑΡΧΙΤΕΚΤΟΝΙΚΗ

Η διαδρομή μιας μέτρησης



Τα (b) έως (f) σας δίνονται.
Εσείς γράφετε το (g) και το ελέγχετε με το (h).

ΣΑΣ ΔΙΝΟΝΤΑΙ ΕΤΟΙΜΑ

`lunix-module.c`
`lunix-ldisc.{c,h}`
`lunix-protocol.{c,h}`
`lunix-sensors.c · lunix.h`
`mk-lunix-lookup.c`
`lunix-attach.c · mk-lunix-devs.sh`

ΓΡΑΦΕΤΕ ΕΣΕΙΣ

`lunix-chrdev.{c,h}`

Υπάρχει ήδη σκελετός με σχόλια στα σημεία που χρειάζονται κώδικα.

ΣΥΛΛΟΓΗ ΔΕΔΟΜΕΝΩΝ · ΣΑΣ ΔΙΝΕΤΑΙ

Η line discipline είναι ένα φίλτρο που αλλάζει

Η συνήθης, η **N_TTY**, περνά τα bytes στο /dev/ttyUSB1. Το PPP, για παράδειγμα, βάζει τη δική του και τα κάνει πακέτα δικτύου.

Το Lunix καταχωρεί δική του με **tty_register_ldisc()**. Το **lunix-attach** την ενεργοποιεί στη θύρα με ένα ioctl().



ΣΥΛΛΟΓΗ ΔΕΔΟΜΕΝΩΝ · ΣΑΣ ΔΙΝΕΤΑΙ

Από ροή bytes σε μετρήσεις

1

Bytes από την ldisc

`lunix_protocol_received_buf()`

2

Μηχανή καταστάσεων

Μαζεύει bytes μέχρι να γίνει πλήρες πακέτο.

3

Τρεις τιμές uint16_t

Little-endian στο πακέτο: le16_to_cpu().

4

Στον buffer του αισθητήρα

`lunix_sensor_update()`

Όλα αυτά τρέχουν σε **interrupt context**, όταν φτάνουν δεδομένα, ασύγχρονα με κάθε read() των διεργασιών.

ΣΥΛΛΟΓΗ ΔΕΔΟΜΕΝΩΝ · ΣΑΣ ΔΙΝΕΤΑΙ

Οι sensor buffers: η διεπαφή σας

Ένας ανά αισθητήρα (lunix.h)

```
struct lunix_sensor_struct {
    struct lunix_msr_data_struct
        *msr_data[N_LUNIX_MSR];
    spinlock_t lock;
    wait_queue_head_t wq;
};

struct lunix_msr_data_struct {
    uint32_t magic;
    uint32_t last_update;
    uint32_t values[];
};
```

Ο producer (lunix-sensors.c)

```
lunix_sensor_update(s, batt, temp, light)
{
    spin_lock(&s→lock);
    /* γράψει values[0] και */
    /* last_update σε κάθε msr */
    spin_unlock(&s→lock);

    wake_up_interruptible(&s→wq);
}
```

Εσείς είστε ο **consumer**: παίρνετε το ίδιο lock για να διαβάσετε, και κοιμάστε στην ίδια **wq**.

ΣΥΛΛΟΓΗ ΔΕΔΟΜΕΝΩΝ · ΣΑΣ ΔΙΝΕΤΑΙ

Στον πυρήνα δεν κάνουμε πράξεις float

Η μετατροπή της ακατέργαστης τιμής σε °C ή Volt θέλει διαιρέσεις και λογαρίθμους. Όμως:

Η FPU δεν σώζεται

Οι καταχωρητές της ανήκουν στη διεργασία που διακόψαμε.

Δεν υπάρχει παντού

Κάποιες αρχιτεκτονικές το κάνουν σε λογισμικό, που λείπει από τον πυρήνα.

Λύση: πίνακες αναζήτησης, υπολογισμένοι από πριν

```
/* lunix-lookup.h: το παράγει το make,
   τρέχοντας το mk-lunix-lookup */
long lookup_temperature[65536];

/* raw: 16 bits από τον buffer */
long v = lookup_temperature[raw];
/* v == 27791, δηλαδή 27.791 × 1000 */

snprintf(buf, sz, "%ld.%03ld\n",
         v / 1000, v % 1000);
```

Ένας ακέραιος ανά δυνατή τιμή 16 bit. Προσοχή στο **πρόσημο** για αρνητικές θερμοκρασίες.

Η ΣΥΣΚΕΥΗ ΧΑΡΑΚΤΗΡΩΝ · ΤΟ ΔΙΚΟ ΣΑΣ ΚΟΜΜΑΤΙ

Ο minor number λέει ποιος αισθητήρας, ποια μέτρηση

```
minor = αισθητήρας × 8 + μέτρηση
```

αρχείο	major	minor
/dev/lunix0-batt	60	0
/dev/lunix0-temp	60	1
/dev/lunix0-light	60	2
/dev/lunix1-temp	60	9
/dev/lunix2-light	60	18

Καταχώρηση, μία φορά στην init

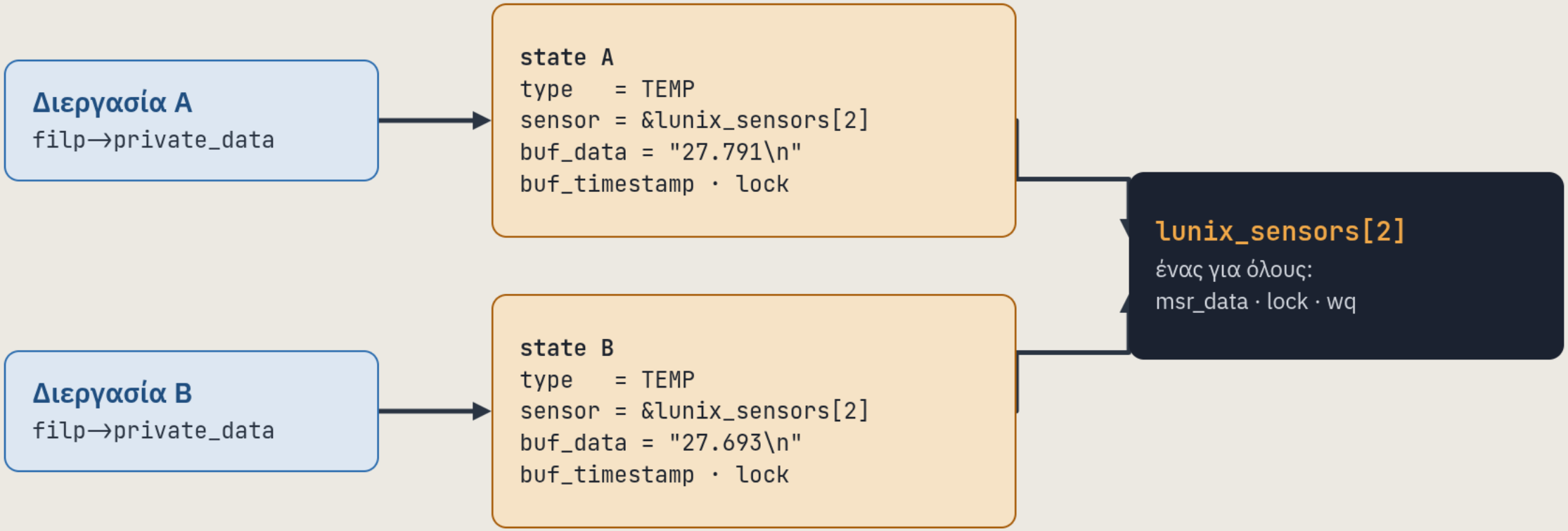
```
dev_no = MKDEV(60, 0);
register_chrdev_region(dev_no,
    16 * 8, "lunix");
cdev_add(&lunix_chrdev_cdev,
    dev_no, 16 * 8);
```

Αποκωδικοποίηση, σε κάθε open()

```
minor  = iminor(inode);
sensor = minor / 8;   /* minor >> 3 */
type   = minor % 8;   /* minor & 7  */
```

Η ΣΥΣΚΕΥΗ ΧΑΡΑΚΤΗΡΩΝ · ΤΟ ΔΙΚΟ ΣΑΣ ΚΟΜΜΑΤΙ

Κάθε open() έχει τη δική της κατάσταση



Η ΣΥΣΚΕΥΗ ΧΑΡΑΚΤΗΡΩΝ · ΤΟ ΔΙΚΟ ΣΑΣ ΚΟΜΜΑΤΙ

Η read() είναι η καρδιά της υλοποίησης

01	Κλείδωσε την κατάσταση αυτού του ανοιχτού αρχείου.	`down_interruptible(&state→lock)`
02	Αν **f_pos == 0**: φέρε νέα τιμή. Αν δεν υπάρχει, **κοιμήσου** μέχρι να έρθει.	`lunix_chrdev_state_update()`
03	Δώσε στον χρήστη bytes από τη θέση f_pos, έως το τέλος της τιμής.	`copy_to_user(usrbuf, buf_data + *f_pos, n)`
04	Προχώρησε το f_pos. Αν τελείωσε η τιμή, γύρνα το στο 0.	`if (*f_pos == buf_lim) *f_pos = 0;`
05	Ξεκλείδωσε και επέστρεψε πόσα bytes έδωσες.	`up(&state→lock); return n;`

Η ΣΥΣΚΕΥΗ ΧΑΡΑΚΤΗΡΩΝ · ΤΟ ΔΙΚΟ ΣΑΣ ΚΟΜΜΑΤΙ

Ύπνος χωρίς να κρατάμε το lock

```
/* μέσα στη read(), με το state→lock κλειδωμένο */
if (*f_pos == 0) {
    while (lunix_chrdev_state_update(state)
            == -EAGAIN) {
        up(&state→lock);
        if (filp→f_flags & O_NONBLOCK)
            return -EAGAIN;
        if (wait_event_interruptible(sensor→wq,
                lunix_chrdev_state_needs_refresh(state)))
            return -ERESTARTSYS;
        if (down_interruptible(&state→lock))
            return -ERESTARTSYS;
    }
}
```

up() πριν τον ύπνο: αλλιώς μια άλλη διεργασία με το ίδιο fd κολλάει πίσω μας.

while, όχι if: όταν ξυπνήσουμε, κάποιος άλλος μπορεί να πρόλαβε.

-ERESTARTSYS: ήρθε σήμα, π.χ. Ctrl-C. Ο πυρήνας αναλαμβάνει τα υπόλοιπα.

Η συνθήκη δεν κοιμάται και δεν παίρνει σημαφόρο: τρέχει μέσα στη wait_event.

Η ΣΥΣΚΕΥΗ ΧΑΡΑΚΤΗΡΩΝ · ΤΟ ΔΙΚΟ ΣΑΣ ΚΟΜΜΑΤΙ

«Υπάρχει κάτι καινούριο;» Το λένε τα timestamps

needs_refresh: μια σύγκριση

```
struct lunix_msr_data_struct *m =
    state→sensor→msr_data[state→type];

return m→last_update
       ≠ state→buf_timestamp;
```

Είναι η συνθήκη της wait_event: πρέπει να είναι γρήγορη και να μην κοιμάται.

update: λίγο μέσα στο spinlock

```
spin_lock_irqsave(&sensor→lock, flags);
raw = m→values[0];
ts  = m→last_update;
spin_unlock_irqrestore(&sensor→lock, flags);

if (ts == state→buf_timestamp)
    return -EAGAIN;
/* εκτός lock: lookup + snprintf */
state→buf_timestamp = ts;
```

Μέσα στο spinlock μόνο αντιγραφή. Η μορφοποίηση γίνεται μετά.

Η ΣΥΣΚΕΥΗ ΧΑΡΑΚΤΗΡΩΝ · ΤΟ ΔΙΚΟ ΣΑΣ ΚΟΜΜΑΤΙ

Ποιο lock προστατεύει τι

Δεδομένα	Ποιος τα αγγίζει	Context	Μηχανισμός
Sensor buffer	lunix_sensor_update και η read σας	ασύγχρονο + διεργασίας	spinlock (irqsave)
Κατάσταση αρχείου	διεργασίες που μοιράζονται το fd	διεργασίας	σημαφόρος
Αναμονή	η read σας ↔ ο producer	διεργασίας ↔ ασύγχρονο	wait queue

⚠ Με spinlock δεν κοιμόμαστε: ούτε copy_to_user, ούτε down, ούτε kmalloc(GFP_KERNEL).

Η ΣΥΣΚΕΥΗ ΧΑΡΑΚΤΗΡΩΝ · ΤΟ ΔΙΚΟ ΣΑΣ ΚΟΜΜΑΤΙ

Μια τιμή, πολλές read(): ο ρόλος του f_pos



buf_data · buf_lim = 7

Στο τέλος, f_pos = 0 και η επόμενη read() περιμένει νέα μέτρηση.

dd ... bs=N	τι επιστρέφουν οι διαδοχικές read()
bs=256	"27.791\n" · μετά ύπνος
bs=5	"27.79" · "1\n" · μετά ύπνος
bs=2	"27" · ".7" · "91" · "\n" · μετά ύπνος
bs=1	ένα byte σε κάθε read(), επτά read() ανά τιμή

ΠΡΑΚΤΙΚΑ

Χτίστε το σε δύο στάδια

g1

Μια συσκευή χαρακτήρων που δουλεύει

- init, open, release, read
- επιστρέφει σταθερή, ψεύτικη τιμή
- χωρίς ύπνο, χωρίς sensor buffers

Ελέγχετε: insmod / rmmod καθαρά, cat στο αρχείο.

g2

Πραγματικά δεδομένα

- τιμές από τους sensor buffers
- ύπνος μέχρι νέα μέτρηση
- κλειδώματα για πολλούς αναγνώστες

Ελέγχετε: πραγματικές τιμές, ταυτόχρονες cat / dd.

ΠΡΑΚΤΙΚΑ

Το δικό σας VM, με αληθινά δεδομένα

Το VM σας · με root
`/dev/ttyS1 → lunix-attach`

⬇

Serial chardev socket
μια σειριακή του VM πάνω από TCP

⬇

`lunix.cslab.ece.ntua.gr:49152`
εκπέμπει συνεχώς μετρήσεις αισθητήρων

Παράδειγμα με QEMU

```
$ qemu-system-x86_64 -enable-kvm ... \
  -chardev socket,id=sensors0,\
    host=lunix.cslab.ece.ntua.gr,\
    port=49152 \
  -device isa-serial,chardev=sensors0

# μέσα στο VM: ποια ttyS είναι;
# dmesg | grep ttyS
```

Άλλος hypervisor; Αρκεί μια σειριακή του VM συνδεδεμένη σε TCP socket.

Ο κώδικας είναι για **Linux 7.3**: πρώτο βήμα, μεταγλώττιση και εγκατάσταση αυτού του πυρήνα μέσα στο VM σας.

ΠΡΑΚΤΙΚΑ

Ο κύκλος ανάπτυξης

ΤΕΡΜΑΤΙΚΟ 1 · ΧΡΗΣΤΗΣ

```
$ vim lunix-chrdev.c
$ make
```

ΤΕΡΜΑΤΙΚΟ 2 · ROOT

```
# rmmod lunix; insmod ./lunix.ko
# ./lunix-attach /dev/ttyS1
```

ΤΕΡΜΑΤΙΚΟ 3 · LOGS

```
# dmesg -w
# less +F /var/log/kern.log
```

printk() παντού

Ο σκελετός έχει ήδη μια μακροεντολή debug() για αυτό.

Διαβάστε τον κώδικα

`elixir.bootlin.com/linux/v7.3`

Ένα Oops δεν είναι καταστροφή

Είστε σε VM: διαβάστε το stack trace, κάντε reboot, συνεχίστε.

ΠΡΑΚΤΙΚΑ

Πώς δείχνετε ότι δουλεύει

Κάθε αισθητήρας, κάθε μέτρηση

```
$ cat /dev/lunix0-batt
```

Ταυτόχρονοι αναγνώστες

```
$ dd if=/dev/lunix0-temp bs=N
```

σε δύο τερματικά, για N = 1, 2, 5, 256

Τι ακριβώς συμβαίνει

```
$ strace dd if=... bs=5
```

εξηγήστε κάθε read() και την τιμή της

(h) Ένα δικό σας πρόγραμμα

Διαβάζει περιοδικά, εμφανίζει και καταγράφει τις μετρήσεις σε αρχείο.

ΠΡΟΑΙΡΕΤΙΚΑ

Για όσους θέλουν παραπάνω

`ioctl()`

Raw και cooked τρόπος

Cooked: μορφοποιημένες τιμές, όπως τώρα.
Raw: τα 16 bit όπως ήρθαν· τη μετατροπή την κάνει το user space.

`.unlocked_ioctl`

`mmap()`

Μετρήσεις χωρίς κλήση συστήματος

Ο sensor buffer απεικονίζεται απευθείας, μόνο για ανάγνωση, στη μνήμη της διεργασίας.

Υπόδειξη: η απεικόνιση αφορά πάντα ολόκληρες σελίδες.

ΥΛΙΚΟ ΚΑΙ ΕΠΙΚΟΙΝΩΝΙΑ

Καλή επιτυχία!

Της άσκησης

Εκφώνηση
Οδηγός της άσκησης
Σκελετός κώδικα lunix-tng

Linux Device Drivers, 3η

κεφ. 3: scull
κεφ. 5–6: συγχρονισμός, ύπνος
κεφ. 15: mmap · κεφ. 18: TTY

Ο ίδιος ο πυρήνας

`elixir.bootlin.com/linux/v7.3`
`docs.kernel.org`

Ερωτήσεις και προβλήματα: τι τρέξατε, τι περιμένατε, τι είδατε.

`cslab-assist@cslab.ece.ntua.gr`